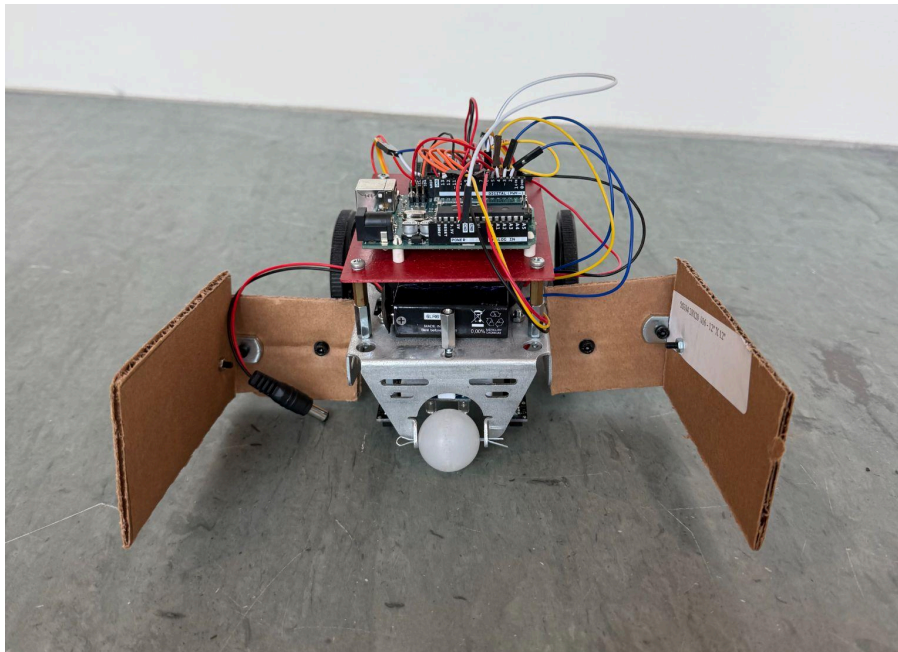


Group 55

Ravi Myers - Rjm479, Liam Bayne - Lrb88, Dan O'Malley - dbo33

1. Robot Design and Strategy Overview

The robot is built on the standard provided chassis and is driven by two Parallax Continuous Rotation Servo motors, one on each side, controlled through L9110H H-bridge drivers. An Arduino UNO serves as the central controller, running all code in C using registers as required. Power is supplied by a four-AA battery pack for the motors and a separate 9V battery for the Arduino.



[Figure 1 - Front Robot View]

The sensing system uses two QTI sensors and one color sensor. The two QTI sensors are mounted on the left and right sides of the robot's front edge. They detect the black border around the competition field, allowing the robot to identify when it is approaching or crossing the boundary and react accordingly by reversing direction. The color sensor is mounted at the center underside of the robot and determines which colored zone (blue or yellow) the robot is placed on, allowing it to identify the opponent's side and respond accordingly.

The robot's strategy is straightforward by design. It begins each match angled toward one of the front corners of its starting zone, driving forward to collect cubes that accumulate along the board's edge. When either QTI sensor detects the black border, the robot reverses and steers back toward the center of the field to gather additional cubes. This edge-sweeping motion allows the robot to continuously cycle through the

cube-dense areas near the borders and ideally stop and protect its cubes against a black border.

The cube-gathering mechanism consists of two L-shaped cardboard arms extending outward from both sides of the chassis. These arms form a wide enclosure around the front of the robot, gathering cubes within the robot's perimeter as it drives forward. The arms are attached to the chassis using bolts and wing nuts, making them adjustable and replaceable. This minimalist mechanical design proved highly effective: the wide sweep of the arms captures a large number of cubes per pass with very little mechanical complexity or potential for failure.

2. Design Process Reflection

From the outset, we prioritized simplicity and reliability over mechanical complexity. Rather than pursuing a sophisticated arm mechanism or an elaborate sensing algorithm, the focus was on building something robust that would function consistently. The L-shaped cardboard arms were selected because they are inexpensive and easy to repair or replace in the field.

We progressed through the four project milestones steadily, generally completing each one one to two days before the deadline and finalizing the demonstration with a TA on the day the milestone was due. Milestone 1, the design pitch, described the core strategy that we carried through to the final robot with minimal deviation. Milestone 2 (mobility) and Milestone 3 (color sensing and border detection) were completed without significant difficulty, as the sensor integration was relatively straightforward once the QTI threshold values were calibrated for the competition field surface. Milestone 4 (cube clearing) validated that the arm geometry and driving pattern were sufficient to gather the required number of cubes within the time limit.

The most significant challenge arose the day before competition, when the robot's software algorithm began behaving unexpectedly. While it had functioned during our testing the previous few days, it suddenly was failing due to some wiring that failed. We identified and resolved the issues that evening. This experience reinforced the value of our minimalist philosophy. The lack of complicated systems allowed us to repair far easier than other teams and reduced the risk of random extraneous tricks breaking.

3. Competition Analysis

The robot finished the round-robin stage with a record of three wins, two losses, and one tie, placing third in its group. This result narrowly missed advancement to the single-elimination bracket.

The first match ended in a tie after both robots became entangled and pushed each other off the side of the board, preventing either from accumulating a meaningful cube count. The following two matches were wins, demonstrating that the edge-sweeping

strategy was effective under normal conditions. One notable moment during the winning matches involved an opposing team that deployed a large horizontal barrier as part of their robot's mechanism, which made contact with and damaged the right arm of the robot. This hampered our performance, but did not stop our robot, as it was reasonably fortified enough to prevent significant damage.

The two losses shared a common root cause: the L-shaped arms extended far enough in front of the robot that, when the QTI sensors detected the black border and triggered a reversal, the tips of the arms were close to over the edge of the board. This created an opportunity for cubes near the front of the enclosure to slide out from under the arms and off the board before the robot could complete its turn, especially if the robots didn't clash and push each other off. In most cases the robot retained the majority of its gathered cubes, but in the two matches where losses occurred, the margin was narrow enough that the lost cubes were decisive.

The arm design demonstrated clear strengths: it was simple and effective at sweeping cubes. The primary weakness was the sensor-to-arm offset, which caused the border-detection response to lag behind the physical position of the arms.

4. Conclusions

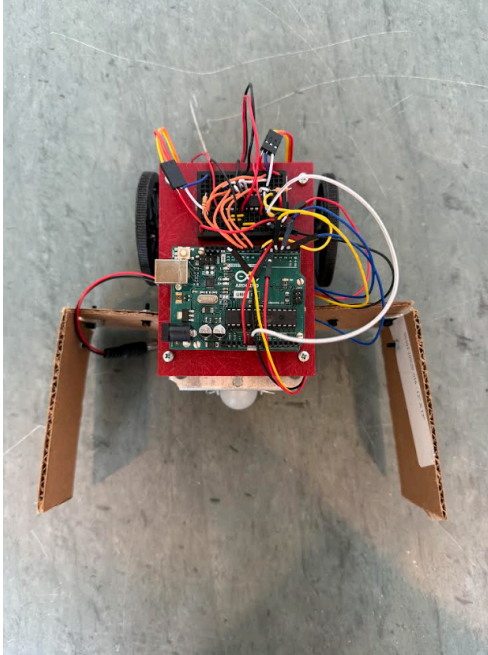
The robot performed competitively and demonstrated that a minimalist design approach can be effective in this type of competition. The core strategy of edge-sweeping with wide cardboard arms required no additional purchased parts beyond basic fasteners and worked reliably across multiple matches.

The primary change that would be made with another attempt at the project is repositioning the QTI sensors further forward on the chassis, ideally at or beyond the leading edge of the arms or pushing the arms back so that they This would ensure that border detection triggers before the arm tips reach the edge of the board, preventing cube loss during the reversal maneuver.

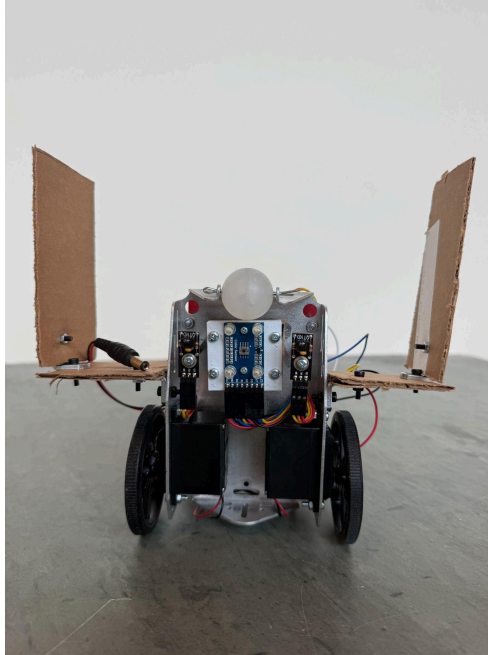
A secondary improvement would be to increase the rigidity of the arm attachment points. While the bolt-and-wing-nut connection made for easy field repairs, the arms occasionally shifted out of alignment during matches, particularly after contact with other robots. A more robust bracket or a backing plate behind each arm would reduce this issue.

For students taking on this project in future years, our primary advice is to resist overcomplicating the design. Ambitious mechanical systems and elaborate algorithms introduce more failure points, and a robot that functions reliably with a simple strategy will almost always outperform one with a sophisticated strategy that breaks down under competition conditions. Start testing the full integrated system early, and set aside dedicated time the day before competition for a complete end-to-end test run, as issues discovered at that stage are still recoverable.

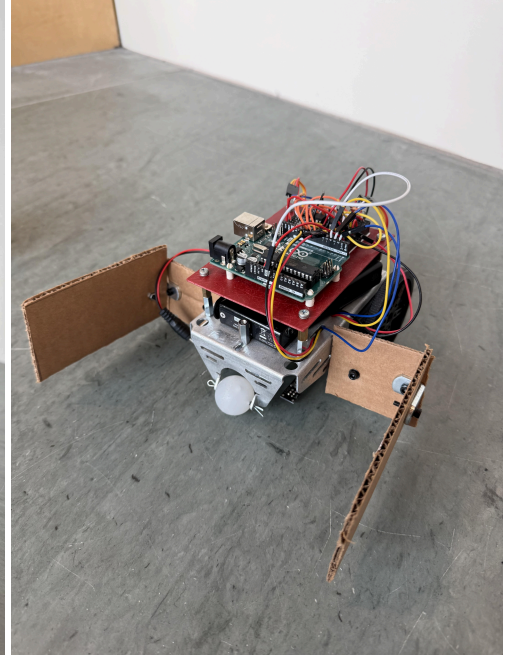
Photos



[Figure 2 - Top Robot View]



[Figure 3 - Bottom Robot View]



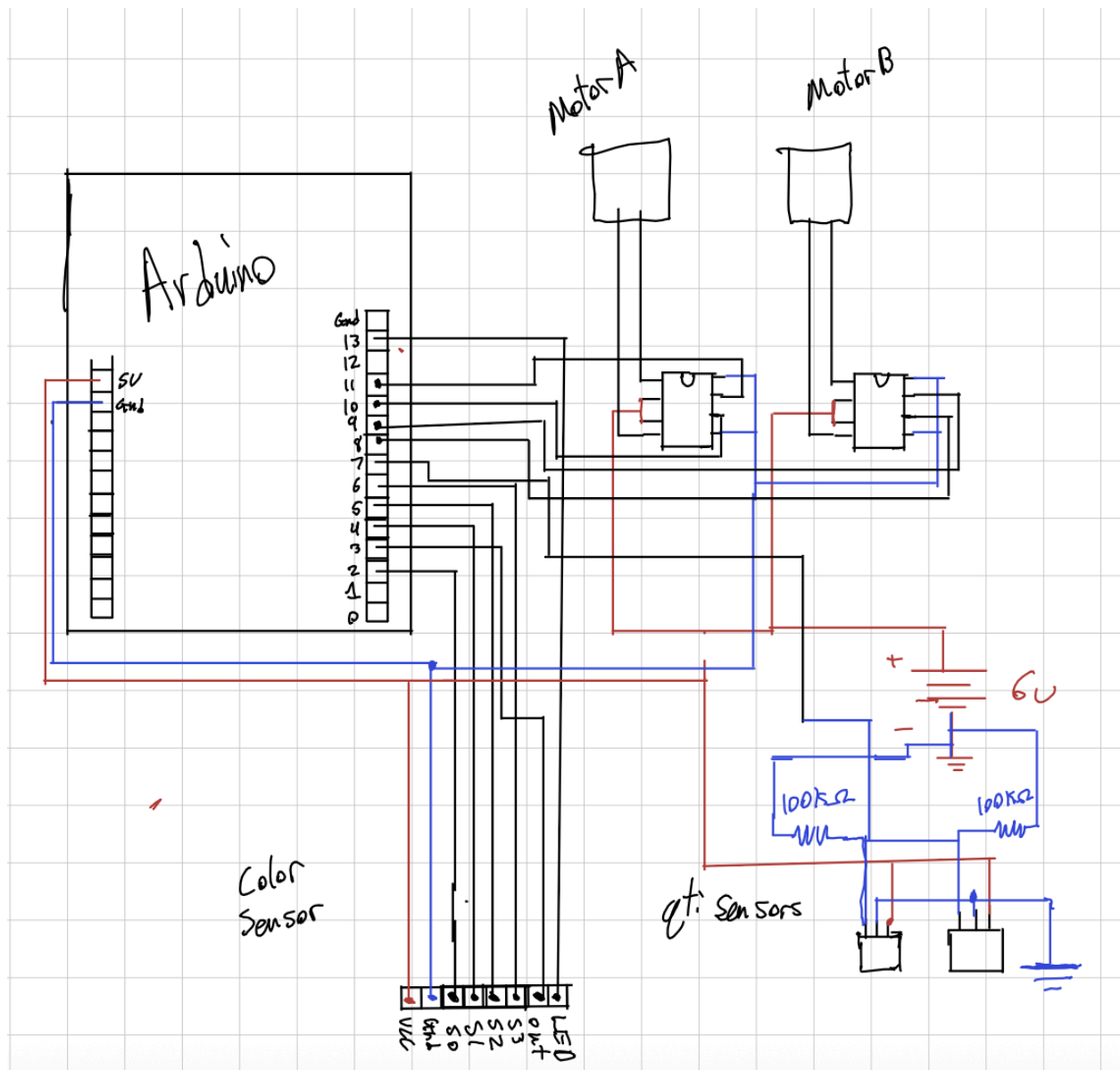
[Figure 4 - Side Robot View]

Appendix A

Bill Of Materials				
Item	Component name	Quantity	Cost	Notes
1	3' by 2' cardboard	1	\$0.00	Found in Mechatronics Lab
2	Brackets	4	\$0.00	Found in Mechatronics Lab
3	M3 bolts/nuts	4	\$0.00	Found in Mechatronics Lab
Final Leftover Budget: \$40.00				

[Table 1 - Bill Of Materials]

Appendix B

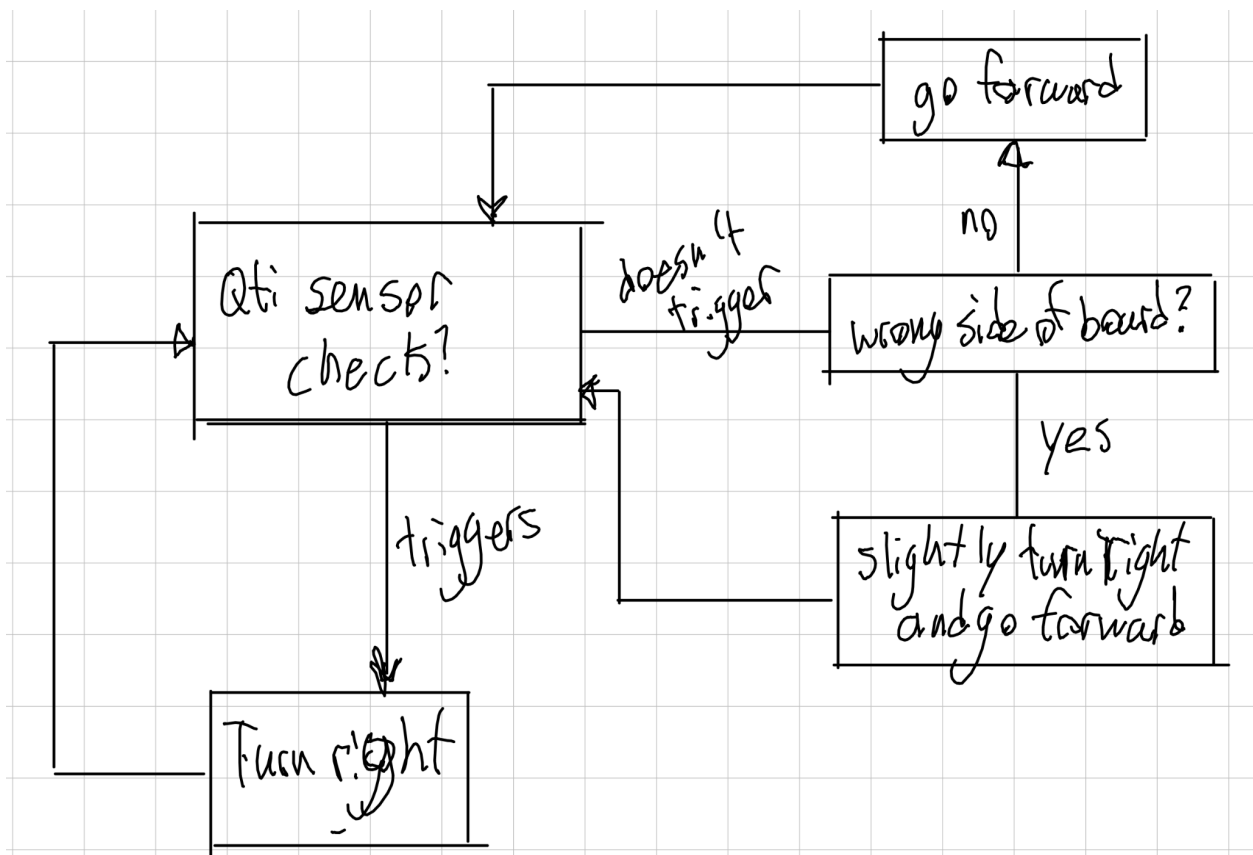


[Figure 5 - Wiring Diagram]

Appendix C

We have no CAD files or drawings that were used for manufacturing individual parts. It was unnecessary for us to weigh our cardboard or bolts because they added negligible weight compared to the preexisting robot.

Appendix D



[Figure 1 - High Level Algorithm]

Appendix E

Motor wiring:

PB0 pin 8 = H-bridge 1 IA

PB1 pin 9 = H-bridge 1 IB

PB2 pin 10 = H-bridge 2 IA

PB3 pin 11 = H-bridge 2 IB

QTI wiring:

Left QTI -> D7 = PD7

Right QTI -> D12 = PB4

*/

// Initial sensing time

#define STARTUP_DELAY_MS 100

// Sets forward drive time to leave starting border

#define START_CLEAR_MS 160

// Sets short forward movement time between sensor checks

#define FORWARD_STEP_MS 60

// Sets pause time before checking sensors again

#define SENSE_PAUSE_MS 15

// Sets backup time after first border detection

#define FIRST_BACKUP_MS 80

// Sets turn time after first border detection

#define FIRST_TURN_MS 200

// Sets short forward movement after first recovery turn

#define FIRST_FORWARD_COMMIT_MS 40

// Sets backup time during final-drive recovery

#define FINAL_RECOVER_BACK_MS 60

// Sets turn time during final-drive recovery

#define FINAL_RECOVER_TURN_MS 70

```

// Sets forward commit time after final-drive recovery
#define FINAL_RECOVER_FORWARD_MS 40

// Sets pause before checking sensors again near final border
#define FINAL_RECHECK_MS 40

/* Motor functions */

// Defines function to drive both motors forward for t milliseconds
void drive_forward_delay(unsigned int t)
{
    // Clears PB0-PB3, then sets PB0 and PB2 high for forward motion
    PORTB = (PORTB & 0b11110000) | 0b00000101;

    // Repeats once per millisecond until t reaches zero
    while (t--)
    {
        // Delays for 1 ms each loop
        _delay_ms(1);
    }
}

// Defines function to stop both motors for t milliseconds
void stop_motors_delay(unsigned int t)
{
    // Clears PB0-PB3 so all motor control pins are low
    PORTB = (PORTB & 0b11110000) | 0b00000000;

    while (t--)
    {
        _delay_ms(1);
    }
}

```



```
// Defines function to drive both motors backward for t milliseconds
void drive_backward_delay(unsigned int t)
{
    // Clears PB0-PB3, then sets PB1 and PB3 high for backward motion
    PORTB = (PORTB & 0b11110000) | 0b00001010;

    while (t--)
    {
        _delay_ms(1);
    }
}

// Defines function to turn robot right for t milliseconds
void drive_right_delay(unsigned int t)
{
    // Clears PB0-PB3, then sets motor pins for right turning
    PORTB = (PORTB & 0b11110000) | 0b00001001;

    while (t--)
    {
        _delay_ms(1);
    }
}

// Defines function to turn robot left for t milliseconds
void drive_left_delay(unsigned int t)
{
    // Clears PB0-PB3, then sets motor pins for left turning
    PORTB = (PORTB & 0b11110000) | 0b00000110;

    while (t--)
    {
        _delay_ms(1);
    }
}
```

```

}
}

/* QTI functions */

// Defines function to initialize the QTI sensor pins
void initQTI(void)
{
    // Left QTI on D7 / PD7 as input

    // Clears PD7 so Arduino pin D7 is an input
    DDRD = DDRD & 0b01111111;

    // Clears PD7 output latch so internal pull-up is off
    PORTD = PORTD & 0b01111111;

    // Right QTI on D12 / PB4 as input

    // Clears PB4 so Arduino pin D12 is an input
    DDRB = DDRB & 0b11101111;

    // Clears PB4 output latch so internal pull-up is off
    PORTB = PORTB & 0b11101111;
}

// Defines function to read the left QTI sensor
int getQTILeft(void)
{
    // Checks whether PD7 is high
    if (PIND & 0b10000000)
    {
        // Returns 1 if left sensor sees black
        return 1;
    }
}

```

```

else
{
    // Returns 0 if left sensor sees color
    return 0;
}
}

// Defines function to read the right QTI sensor
int getQTIRight(void)
{
    // Checks whether PB4 is high
    if (PINB & 0b00010000)
    {
        // Returns 1 if right sensor sees black
        return 1;
    }
    else
    {
        // Returns 0 if right sensor sees color
        return 0;
    }
}

/* Helper functions */

// Defines function that permanently stops the robot
void stopForever(void)
{
    // Creates an infinite loop
    while (1)
    {
        // Clears PB0-PB3 so both motors stay stopped
        PORTB = (PORTB & 0b11110000) | 0b00000000;
    }
}

```

```

}

// Stop permanently only when both sensors see black

// Defines function to check whether both sensors see black
void checkFinalStop(void)
{
    // Stores current left QTI reading
    int leftBlack = getQTILeft();

    // Stores current right QTI reading
    int rightBlack = getQTIRight();

    // Checks whether both sensors detect black
    if (leftBlack && rightBlack)
    {
        // Stops motors for 1000 ms before permanent stop
        stop_motors_delay(1000);

        // Enters infinite stopped state
        stopForever();
    }
}

// stop -> back up -> turn away -> short forward

// Defines recovery behavior after first border contact
void recoverFromFirstBorder(int leftBlack, int rightBlack)
{
    // Briefly stops before backing up
    stop_motors_delay(30);

    // Backs up away from the border
    drive_backward_delay(FIRST_BACKUP_MS);
}

```

```
// Briefly stops before turning
stop_motors_delay(30);

// Checks if only the left sensor hit black
if (leftBlack && !rightBlack)
{
    // Left sensor hit black, turn right

    // Turns right away from left-side border contact
    drive_right_delay(FIRST_TURN_MS);

    // Briefly stops after turning
    stop_motors_delay(30);
}

// Checks if only the right sensor hit black
else if (rightBlack && !leftBlack)
{
    // Right sensor hit black, turn left

    // Turns left away from right-side border contact
    drive_left_delay(FIRST_TURN_MS);

    // Briefly stops after turning
    stop_motors_delay(30);
}
else
{
    // If both hit black, default to slight right

    // Uses right turn as default recovery turn
    drive_right_delay(FIRST_TURN_MS);
}
```

```
// Briefly stops after turning
stop_motors_delay(30);
}

// Moves forward slightly to leave the border region
drive_forward_delay(FIRST_FORWARD_COMMIT_MS);

// Briefly stops after forward commit
stop_motors_delay(30);
}

// one sensor black -> likely side-border -> recover and continue
// both sensors black -> final stop

// Defines recovery behavior during final drive
void recoverOnFinalDrive(int leftBlack, int rightBlack)
{
    // Briefly stops before rechecking sensors
    stop_motors_delay(30);

    // Waits briefly before reading sensors again
    stop_motors_delay(FINAL_RECHECK_MS);

    // Updates left QTI reading
    leftBlack = getQTILeft();

    // Updates right QTI reading
    rightBlack = getQTIright();

    // Checks whether both sensors now see black
    if (leftBlack && rightBlack)
    {
        // Stops motors for 1000 ms before permanent stop
        stop_motors_delay(1000);
    }
}
```

```
// Enters infinite stopped state
stopForever();
}

// Backs up away from possible side-border contact
drive_backward_delay(FINAL_RECOVER_BACK_MS);

// Briefly stops before turning
stop_motors_delay(30);

// Checks if only the left sensor sees black
if (leftBlack && !rightBlack)
{
    // Left side touched black, turn right

    // Turns right away from left-side contact
    drive_right_delay(FINAL_RECOVER_TURN_MS);

    // Briefly stops after turning
    stop_motors_delay(30);
}

// Checks if only the right sensor sees black
else if (rightBlack && !leftBlack)
{
    // Right side touched black, turn left

    // Turns left away from right-side contact
    drive_left_delay(FINAL_RECOVER_TURN_MS);

    // Briefly stops after turning
    stop_motors_delay(30);
}
```

```

else
{
    // If uncertain, do a small default right turn

    // Uses right turn as default recovery turn
    drive_right_delay(FINAL_RECOVER_TURN_MS);

    // Briefly stops after turning
    stop_motors_delay(30);
}

// Commit forward so robot actually leaves the border area

// Moves forward slightly after recovery
drive_forward_delay(FINAL_RECOVER_FORWARD_MS);

// Briefly stops after forward commit
stop_motors_delay(30);
}

/* Main */

// Main program entry point
int main(void)
{
    // Declares variable for left QTI black detection
    int leftBlack;

    // Declares variable for right QTI black detection
    int rightBlack;

    // Motor outputs on PB0-PB3

    // Sets PB0-PB3 as outputs while leaving other PORTB pins unchanged

```



```

DDRB |= 0b00001111;

// Motors off initially

// Clears PB0-PB3 so motors begin stopped
PORTB &= 0b11110000;

// Initializes QTI sensor input pins
initQTI();

// Keeps motors stopped briefly at startup
stop_motors_delay(STARTUP_DELAY_MS);

/*
 0: Quickly clear the starting black border
 */

// Drives forward to leave the starting black border
drive_forward_delay(START_CLEAR_MS);

// Briefly stops after clearing the starting border
stop_motors_delay(30);

/*
 1: Drive until first border contact
 */

// Repeats until first border contact is detected
while (1)
{
    // Reads left QTI sensor
    leftBlack = getQTILeft();

    // Reads right QTI sensor

```

```

rightBlack = getQTIIRight();

// Checks whether either sensor sees black
if (leftBlack || rightBlack)
{
    // Runs first-border recovery behavior
    recoverFromFirstBorder(leftBlack, rightBlack);

    // Exits Phase 1 loop after first recovery
    break;
}

// Drives forward for one short step
drive_forward_delay(FORWARD_STEP_MS);

// Pauses briefly before next sensor check
stop_motors_delay(SENSE_PAUSE_MS);
}

/*
Phase 2: Drive toward opposite end both QTI sensors black -> stop
One QTI sensor black -> Milestone 3-style recovery
*/

// Repeats final-drive behavior forever unless stopForever is called
while (1)
{
    // Drives forward for one short step
    drive_forward_delay(FORWARD_STEP_MS);

    // Pauses briefly before reading sensors
    stop_motors_delay(SENSE_PAUSE_MS);

    // Reads left QTI sensor

```

```
leftBlack = getQTILeft();

// Reads right QTI sensor
rightBlack = getQTIRight();

// Checks whether both sensors see black at opposite border
if (leftBlack && rightBlack)
{
    // Stops motors for 1000 ms before permanent stop
    stop_motors_delay(1000);

    // Enters infinite stopped state
    stopForever();
}

// Checks whether only one or either sensor sees black
if (leftBlack || rightBlack)
{
    // Runs side-border recovery or final stop check
    recoverOnFinalDrive(leftBlack, rightBlack);
}
}
}
```